

CMSC451 Midterm 1 Studying

JAMES ZHANG*

October 7, 2024

1. Short Answer Questions

- a) *False*; in the situation that companies make offers to students, once a company makes an offer that is accepted, the company is not guaranteed to remain engaged for the remainder of the algorithm. Suppose c matches with s , but then c' makes an offer to s and they match – company c is no longer matched. On the other hand, if companies are making offers to students, if a student becomes matches, the *student* is guaranteed to remain engaged for the remainder of the algorithm, and the quality of the engagement for the student can only increase $\implies$ **Lemma 1** in Child's notes.
- b) *iii*; the company's preference for the student to whom it is engaged *may either decrease or stay the same from one iteration to the next*.
- c) *True*; (c, s) is an instability.
- d) A counterexample to show that the greedy algorithm does not output a stable matching is

$$\begin{array}{ll} A = \{X, Y\} & X = \{B, A\} \\ B = \{X, Y\} & Y = \{A, B\} \end{array}$$

A will greedily choose X , forcing B to choose Y , but B and X would rather be together.

- e) Running times of certain situations using the Gale-Shapley algorithm
 - All companies have the same preference $\implies \mathcal{O}(n^2)$
 - Each company's top choice is a different student $\implies \mathcal{O}(n)$
- f) $\mathcal{O}(n)$; In the first iteration of the while loop, the for loop iterates n times, then $n/2$, then $n/4, \dots$

$$\sum_{i=0}^{\infty} n \left(\frac{1}{2}\right)^i = n \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i = 2n$$

- g) Time complexity of **ExploreTriangles**

*Email: jzhang72@terpmail.umd.edu

- i. $\mathcal{O}(n * 100 * 100) = \mathcal{O}(n)$
- ii. $\mathcal{O}(n * \sqrt{n} * \sqrt{n}) = \mathcal{O}(n^2)$
- h) Little o, ω, Θ
 - i) More asymptotic stuff
 - j) More asymptotic stuff
 - k) More asymptotic stuff
 - l) More asymptotic stuff
- m) The maximum number of edges in an undirected graph with n vertices, in which each vertex has degree at most k is $\frac{n*k}{2}$.
- n) The number of edges in an n -vertex forest with c components is $n - c$; each component has $c - 1$ edges.
- o) Graph with $4n$ vertices, where each vertex has degree $n + 7 \implies$ the exact number of edges is $\frac{4n(n+7)}{2} = \frac{4n^2+28n}{2} = 2n^2 + 14n \implies \Theta(n^2)$
- p) Different BFS trees example: Consider a graph with 4 edges and 3 nodes. There are edges 12, 13, 34, and 24.
- q) *Not bipartite*; we can arrive at a contradiction at vertex 8.
- r) *Not strongly connected*; no path from 11 to any other vertex because its out-degree is 0.
- s) *False*; every DAG has a topological ordering, but it need not be unique. Consider a graph with 3 nodes, and 2 edges 12, 13. Two valid topological orderings are 1, 2, 3 and 1, 3, 2.
- t) G_2 does not have a valid topological ordering; since there is a cycle in G_1 , simply reversing the edges to obtain G_2 does not eliminate the cycle in G_2 .
- u) *True*; I can't think of a counterexample?
- v) $\{4, 2, 5, 7, 3, 6, 1\}$?
- w) Maximum spanning tree?
- x) *False*

2. Degree sums in undirected graphs.

Suppose $G = (V, E)$ is an undirected graph given in adjacency list format. Recall that this format provides a linked list of the neighbors of each vertex (you should assume that the format does not directly provide the degree of a vertex). For any vertex v , let $\text{sumdegree}[v]$ denote the sum of the degrees of the neighbors of v : e.g., if v has neighbors w_1, w_2 , and w_3 with respective degrees 3, 6, 2, then $\text{sumdegree}[v] = 3 + 6 + 2 = 11$. Show how to construct the array of all $\text{sumdegree}[v]$ values (one value for each $v \in V$), in $O(n + m)$ time. (As usual, we let $n = |V|$ and $m = |E|$.) Explain why your algorithm has the desired running time and why it is correct.

-
- 1 Let degree be a vertex-indexed array where $\text{degree}[v]$ is the degree of vertex v
 - 2 Define a helper function called $\text{DFS}(v)$

```
3  Define a vertex-indexed array called seen representing if a vertex v
    has been explored yet or not
4  DFS(v):
5      if v not in seen:
6          return degree[v]
7      for neighbors in G[v]:
```
